Maximum Bipartite Matching via Augmenting Paths

Workforce allocation with role capacities, Hopcroft–Karp, and a proof of optimality

Delta Code 2026 · Problem Statement 1: Linear Algebra & Graph Optimization

Venkatesh Dharavath · M.Sc. Mathematics & Scientific Computing, NIT Warangal

Greedy hiring leaves people unplaced

2,400
candidates |U|

320
roles |V| (3–10 seats)

6,142
eligibility edges |E|

2,039
total seats Σ c_v

The smallest trap

- Roles X and Y have 1 seat each
- Ann fits X or Y; Ben fits only X
- Greedy gives Ann X, so Ben is stranded: 1 hire
- Ann on Y and Ben on X: 2 hires

The fix

- First-fit decisions are irreversible
- An augmenting path shifts a chain of placed people one step each
- Each chain adds exactly one hire; repeat until no chain exists

Optimal placement beats greedy by 192 hires

1,709

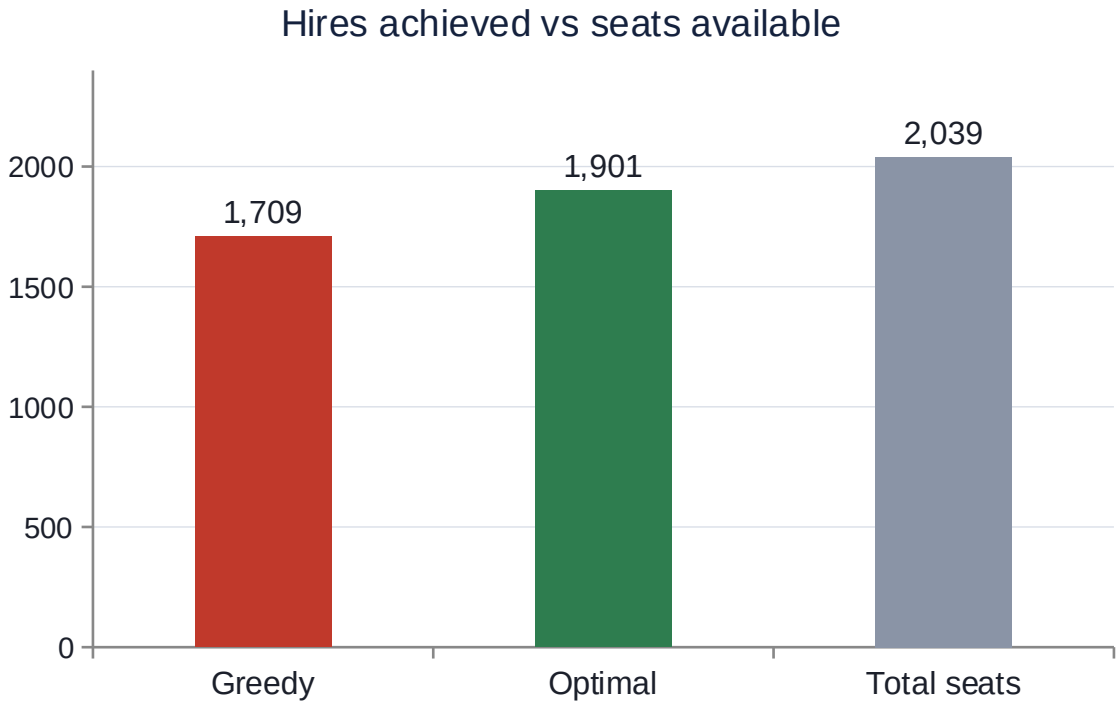
greedy first-fit hires

1,901

optimal hires (Hopcroft–Karp)

+192

Greedy Capacity Gap (+11.2%)



Two equivalent formulations

Flow network G′

- Source s, sink t
- s → u, capacity 1, for each candidate
- u → v, capacity 1, if u is eligible for v
- v → t, capacity c_v (seats)
- Max flow = max hires; an integral flow is a legal assignment

Integer linear program

$$\begin{aligned} \max \quad & \sum x_e \\ \text{s.t.} \quad & \sum x_e \le 1 \quad \text{(candidate } u\text{)} \\ & \sum x_e \le c_v \ \text{(role } v\text{)} \\ & x_e \in \{0,1\} \end{aligned}$$

Each column of A has two 1s (its candidate row, its role row): A is totally unimodular, so the LP relaxation is already integral.

Berge’s Lemma: the optimality test

M is a maximum matching if and only if there is no M-augmenting path (alternating, from a free candidate to a free seat).

(⇒) A path gives a bigger matching

- Flip the path: matched edges out, unmatched edges in
- Both endpoints were free, interior vertices keep one partner
- $|M'| = |M| + 1$, so M was not maximum

(⇐) A bigger matching gives a path

- Take M' with $|M'| > |M|$; look at $M \triangle M'$
- Degree ≤ 2, so it splits into paths and even cycles
- Some odd path has more M' edges; its endpoints are M-free: an augmenting path

Hopcroft–Karp in Dinic form, warm-started

1

Greedy start

Pre-load the greedy assignment as flow, so every fix found repairs a specific greedy mistake

2

BFS levels

Label vertices by distance from s over pipes with spare capacity; if t is unreached, stop: optimal

3

DFS blocking flow

Walk only level k → k+1, push one unit per path, prune dead ends with current-arc pointers

4

Repeat

Shortest path length strictly grows each phase: 5, 7, 9, 11 edges on our data

Only 4 phases were needed from the greedy start (5 from scratch).

Complexity: O(|E|√|V|)

One phase costs O(|E|)

- BFS touches each edge once
- Every edge except (v,t) has capacity 1, so it carries at most one path per phase
- Current-arc pointers: each edge is a dead end at most once
- Total path work ≤ |E| + #paths

About √N phases

- After √N phases the shortest fix has ≥ √N edges
- Candidates have unit throughput, so leftover fixes are disjoint, each using ≥ √N/2 candidates
- At most 2|U|/√N ≤ 2√N remain; each phase fixes ≥ 1
- Observed: 4 phases, far below √N ≈ 52

Code structure and design choices

`generate / load_csv`

Seeded synthetic data or validated real CSVs; original ids preserved

`greedy`

First-fit baseline that defines the gap

`Network`

Hand-written flow graph; edge e and its reverse e^1 stored together

`bfs / blocking_flow`

Iterative level graph + DFS, no recursion limit

`verify + min-cut`

Feasibility check; residual BFS gives the optimality certificate

`ilp_solve / draw`

scipy milp & linprog benchmark; networkx used only for drawing and flow validation

Optimality is certified three independent ways

Berge

1,901

No augmenting path remains: residual BFS cannot reach t

Min-cut

1,901

Capacity of the cut around vertices reachable from s equals the flow

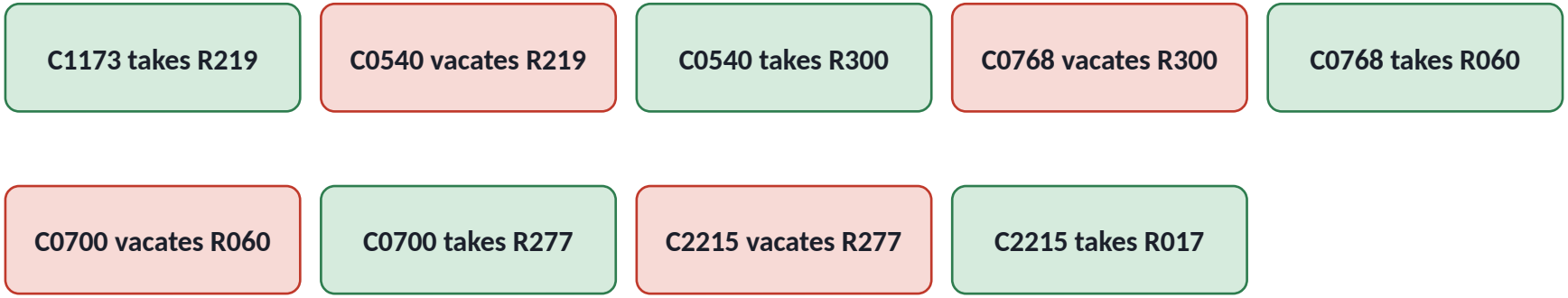
Solvers

1,901

networkx max-flow, scipy milp (ILP) and linprog (LP) all agree

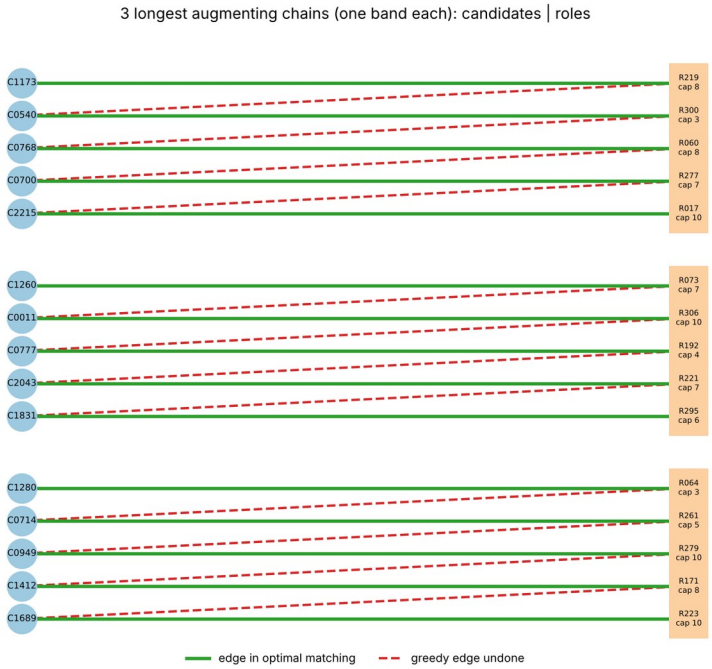
The program asserts all three on every run and aborts on any disagreement.

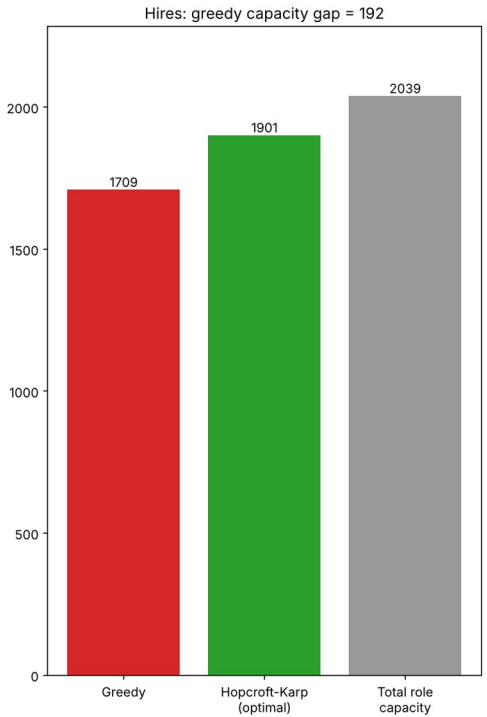
How one greedy mistake is undone



- Green: a new placement. Red: a candidate gives up the seat greedy had given them (a backward residual edge)
- C1173 had no open role because greedy filled R219 early; four earlier hires each shift one step and the chain ends at R017, which still had a free seat
- 5 placements and 4 removals: net +1 hire, nobody loses a job

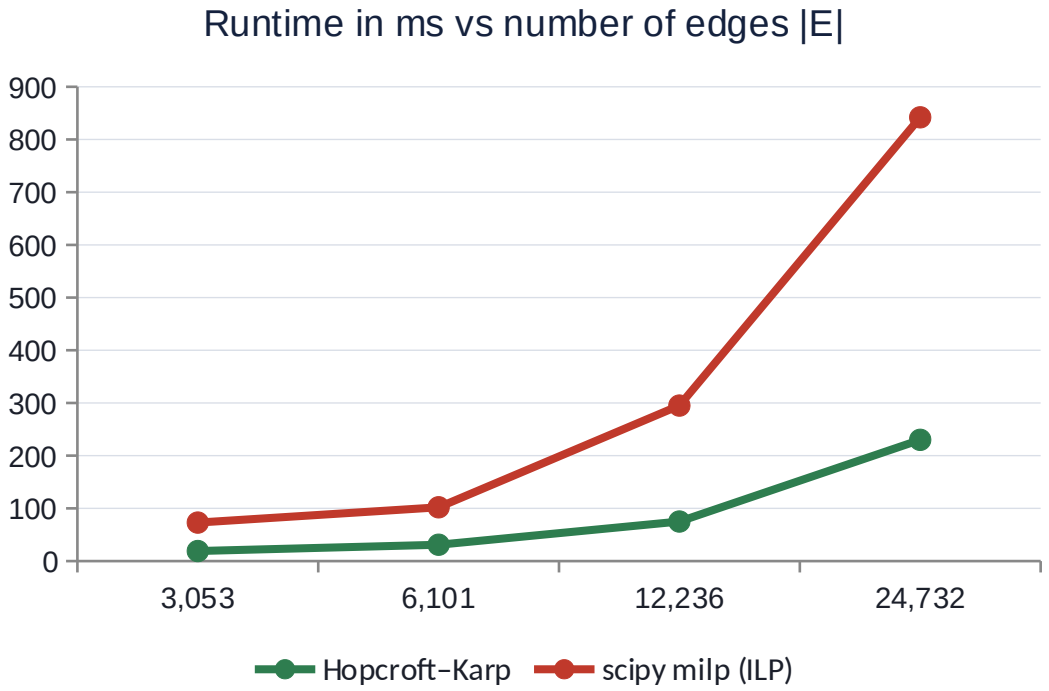
Visualization: chains and the gap





- Green: edge in the optimal matching
- Red dashed: greedy edge that was undone
- Each band is one chain; the staircase is the “everyone shifts one seat” move
- Right panel: the 192-hire gap

Hopcroft–Karp scales gently and beats the ILP



|U|	|E|	HK ms	ILP ms	ILP / HK
1,200	3,053	19	73	**3.8x**
2,400	6,101	31	102	**3.3x**
4,800	12,236	75	295	**3.9x**
9,600	24,732	230	842	**3.7x**

Size ×8.1 (|E|) → HK time ×12, about |E|^1.19, inside the |E|√|V| bound (×23). Main instance: HK 25 ms, LP 35 ms, ILP 123 ms.

Trade-offs between methods

Method	Optimal?	Speed (main instance)	Best used when
Greedy first-fit	No: 1,709	0.4 ms	Instant estimate; never reconsiders
Hopcroft–Karp (ours)	Yes: 1,901	25 ms	Pure matching; provable $O(\|E\|\sqrt{\|V\|})$
LP relaxation	Yes (integral)	35 ms	Extra linear constraints, weights
ILP (milp)	Yes: 1,901	123 ms	Side constraints that break the flow model

Key trade-off: the flow engine is 3–4x faster and carries a proof; the ILP is more general but offers no better answer here.

Constraint adherence and edge cases

Guaranteed on every run

- Only eligible roles assigned
- No role exceeds its capacity
- Warm and cold runs agree
- Min-cut = max-flow = networkx = ILP = LP

8 automated tests

- 300 random small instances vs networkx
- Real string ids kept; duplicate edges collapsed
- Zero-capacity roles; more candidates than seats
- Candidates with no edges; empty input
- Clear errors: unknown role, negative or non-integer capacity, duplicate role, missing column
- A 40-step forced augmenting chain

Takeaways

- Greedy leaves 192 hires (11%) on the table because its choices are irreversible
- Augmenting-path repair recovers all of them in 4 phases, with nobody losing a job
- Berge’s Lemma, a matching min-cut and three solvers certify 1,901 as optimal
- O(|E|√|V|) engine, 3–4x faster than an ILP, tested on edge cases
- Caveat: data is synthetic; run allocate.py with --edges and --roles on real data

Repository: allocate.py · assignment.csv · matching_vis.png · report.md · README.md